DOCUMENTO TÉCNICO

SISTEMA IoT PARA MONITOREO Y ALMACENAMIENTO DE DATOS EN UNA BANDA TRANSPORTADORA INDUSTRIAL

AUTORES

Santiago Amezquita Alfonso

Caren Ximena Sierra Viasus

UNIVERSIDAD PEDAGÓGICA Y TECNOLÓGICA DE COLOMBIA

FACULTAD DE INGENIERÍA

ESCUELA DE INGENIERÍA ELECTRÓNICA

TUNJA

2026

VIGILADA MINEDUCACIÓN

Índice

1. Introducción **2**

2. Descripción del sistema **2**

3. Arquitectura **3**
3.1. Diagrama de flujo: recepción y almacenamiento 4
3.2. Integración de los componentes 5

4. Implementación **5**
4.1. Base de datos 5
4.1.1. Creación de la tabla 6
4.1.2. Inserción de datos 6
4.1.3. Reglas de integridad de los datos 7
4.2. Componentes de software 7
4.3. Consultas de verificación 7
4.3.1. Visualización de todos los registros 7
4.3.2. Cantidad total de registros 8
4.3.3. Consulta de un dispositivo específico 8
4.4. Exportación y visualización de los datos 8
4.4.1. Descarga del archivo de base de datos 8
4.4.2. Exportación a formato CSV 8
4.4.3. Visualización mediante DB Browser for SQLite 9
4.5. Casos de uso 9
4.5.1. Caso de Uso 1: Recepción de datos de un dispositivo ESP32 9
4.5.2. Caso de Uso 2: Verificación de los datos almacenados 10

5. Instalación y configuración **10**
5.1. Requisitos del sistema 10
5.2. Configuración inicial 11
5.3. Compatibilidad y arquitectura del software 11

6. Soporte y mantenimiento **11**
6.1. Actualizaciones 12

7. Estado actual del sistema **12**
7.1. Integración con la plataforma OT-FORTRESS 13

8. Conclusiones **13**

VIGILADA MINEDUCACIÓN

1. Introducción

La implementación de un sistema de Internet de las Cosas (IoT) orientado a un prototipo de una banda transportadora industrial permite integrar dispositivos electrónicos, sensores, sistemas de comunicación y herramientas de almacenamiento de datos para obtener información del proceso .

El sistema desarrollado tiene como propósito adquirir y registrar información generada durante el funcionamiento de una banda transportadora. Para ello se utilizan dispositivos ESP32 encargados de obtener los datos provenientes de los sensores y transmitirlos mediante una red inalámbrica.

La comunicación entre los dispositivos y el sistema de procesamiento se realiza utilizando el protocolo MQTT. Los mensajes son recibidos por un broker Mosquitto instalado en una Raspberry Pi, donde un script desarrollado en Python se encarga de procesar la información y almacenarla en una base de datos SQLite.

Esta arquitectura permite centralizar los datos generados por los diferentes dispositivos IoT, manteniendo un registro organizado de las lecturas recibidas, la fecha y hora de recepción, el dispositivo de origen y el tópico MQTT utilizado.

La solución se plantea como una infraestructura para el almacenamiento y posterior consulta de los datos generados por el sistema de automatización de la banda transportadora. De esta manera, la información recopilada puede ser utilizada posteriormente para tareas de monitoreo, análisis y supervisión del proceso industrial.

Como parte de las siguientes etapas del proyecto, se contempla la integración de estos datos con la plataforma **OT-FORTRESS**, desarrollada para la supervisión de nodos, dispositivos, sensores, alertas y eventos.

2. Descripción del sistema

El sistema IoT desarrollado tiene como función principal recibir, procesar y almacenar la información generada por los dispositivos ESP32 asociados a la banda transportadora.

La arquitectura utiliza procesamiento local mediante una Raspberry Pi, la cual funciona como plataforma de recepción y almacenamiento de los datos. La información se conserva en una base de datos SQLite almacenada localmente.

El flujo general del sistema es:

$$\text{ESP32} \rightarrow \text{Wi-Fi} \rightarrow \text{MQTT} \rightarrow \text{Mosquitto} \rightarrow \text{Python} \rightarrow \text{SQLite}$$

La base de datos utilizada se encuentra almacenada en el archivo:

`iot_seguro.db`

La tabla principal utilizada para almacenar las lecturas recibe el nombre:

`lecturas_iot`

Entre las principales funciones de la implementación se encuentran:

VIGILADA MINEDUCACIÓN

- Recepción de mensajes enviados por los dispositivos ESP32.
- Comunicación mediante el protocolo MQTT.
- Recepción de mensajes mediante el broker Mosquitto.
- Procesamiento de los datos mediante Python.
- Registro de fecha y hora de recepción.
- Identificación del dispositivo de origen.
- Registro del tópico MQTT.
- Almacenamiento de la carga útil recibida.
- Consulta de los registros almacenados.
- Exportación de los datos a formato CSV.

3. Arquitectura

La arquitectura del sistema está compuesta por dispositivos ESP32, una red inalámbrica, un broker MQTT, una Raspberry Pi y una base de datos SQLite.

Cada componente cumple una función específica dentro del proceso de adquisición, comunicación y almacenamiento de la información.

Los principales componentes son:

- **ESP32:** dispositivos utilizados para adquirir o generar información correspondiente al proceso de la banda transportadora.
- **Wi-Fi:** medio de comunicación inalámbrica utilizado para transportar los mensajes generados por los dispositivos.
- **Mosquitto:** broker MQTT encargado de recibir y distribuir los mensajes publicados por los dispositivos.
- **Raspberry Pi:** plataforma utilizada para ejecutar el broker, el script de procesamiento y la base de datos local.
- **Python:** componente encargado de recibir los mensajes MQTT, procesarlos y almacenarlos.
- **SQLite:** sistema de gestión de base de datos utilizado para almacenar localmente los registros.
- **OT-FORTRESS:** plataforma de supervisión prevista para una etapa posterior del proyecto.

La Figura 1 representa la interacción entre los dispositivos de adquisición, la comunicación MQTT, la Raspberry Pi y el sistema de almacenamiento.

VIGILADA MINEDUCACIÓN

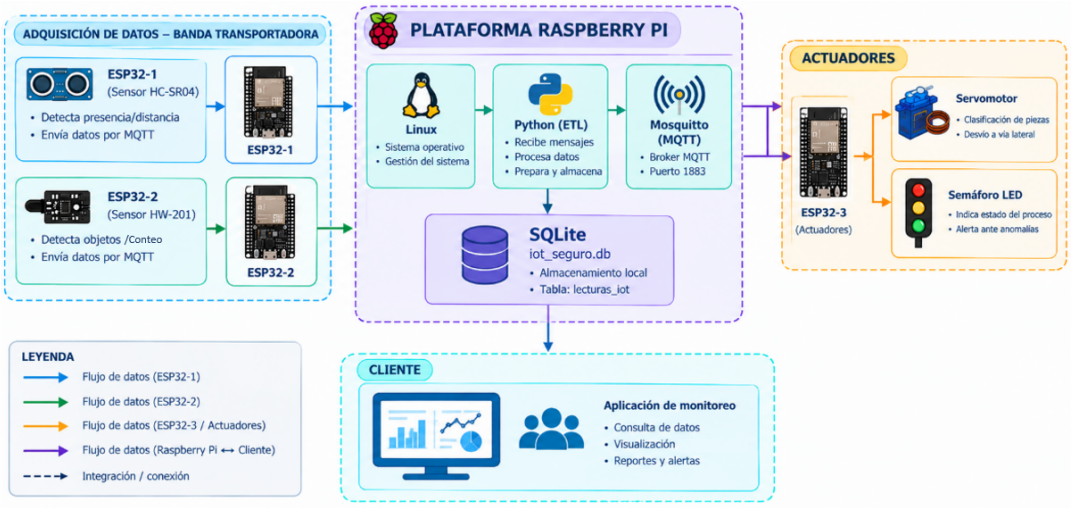


Figura 1: Arquitectura general del sistema IoT

3.1. Diagrama de flujo: recepción y almacenamiento

El proceso inicia cuando uno de los dispositivos ESP32 genera una nueva lectura o evento asociado al funcionamiento de la banda transportadora.

Posteriormente, el dispositivo publica la información utilizando MQTT. El mensaje es recibido por el broker Mosquitto instalado en la Raspberry Pi.

El programa desarrollado en Python mantiene una suscripción al tópico:

```
sensores/#
```

Cuando se recibe un mensaje, la función `on_message()` procesa la información.

El proceso general puede representarse mediante la siguiente secuencia:

$$\boxed{\text{Sensor}} \rightarrow \boxed{\text{ESP32}} \rightarrow \boxed{\text{MQTT}} \rightarrow \boxed{\text{Mosquitto}} \rightarrow \boxed{\text{Python}} \rightarrow \boxed{\text{SQLite}}$$

Durante este procedimiento se realizan las siguientes operaciones:

1. La ESP32 genera un dato.
2. El dispositivo publica el mensaje mediante MQTT.
3. El broker Mosquitto recibe el mensaje.
4. El programa Python recibe el mensaje mediante la suscripción al tópico correspondiente.
5. Se ejecuta automáticamente la función `on_message()`.
6. Python obtiene la fecha y hora del sistema de la Raspberry Pi.
7. Se identifica el dispositivo mediante la información asociada al tópico MQTT.

VIGILADA MINEDUCACIÓN

8. El contenido recibido se convierte de bytes a texto.
9. Los datos son preparados para su almacenamiento.
10. Finalmente, el registro es insertado en la tabla `lecturas_iot`.

3.2. Integración de los componentes

La integración de los elementos del sistema permite establecer una cadena continua desde la adquisición de información hasta su almacenamiento.

En la etapa actual, la infraestructura de almacenamiento está implementada en la Raspberry Pi, mientras que la plataforma OT-FORTRESS se encuentra preparada para una integración posterior.

El flujo completo previsto es:

$$\text{ESP32} \rightarrow \text{MQTT} \rightarrow \text{Raspberry Pi} \rightarrow \text{Python} \rightarrow \text{SQLite} \rightarrow \text{OT-FORTRESS}$$

4. Implementación

La implementación del sistema se realizó utilizando una Raspberry Pi como plataforma central de procesamiento y almacenamiento.

Sobre esta plataforma se instalaron y configuraron los componentes necesarios para la comunicación MQTT y la gestión de la base de datos.

El sistema implementado incluye:

- Sistema operativo Linux en la Raspberry Pi.
- Broker MQTT Mosquitto.
- Script desarrollado en Python.
- Base de datos SQLite.
- Archivo `iot_seguro.db`.
- Tabla `lecturas_iot`.

4.1. Base de datos

La base de datos implementada actualmente contiene una tabla denominada `lecturas_iot`, utilizada para almacenar los mensajes recibidos desde los dispositivos conectados al sistema.

La estructura de la tabla es la siguiente.

VIGILADA MINEDUCACIÓN

Tabla 1: Estructura de la tabla `lecturas_iot`

Campo	Tipo	Restricción	Descripción
`id`	INTEGER	PRIMARY KEY, AUTOINCREMENT	Identificador único de cada registro.
`fecha_hora`	TEXT	NOT NULL	Fecha y hora en la que la Raspberry Pi recibe el mensaje.
`dispositivo`	TEXT	NOT NULL	Identificación del dispositivo que envió la información.
`topico`	TEXT	NOT NULL	Tópico MQTT utilizado para transportar el mensaje.
`carga_util`	TEXT	NOT NULL	Contenido o valor recibido en el mensaje MQTT.

4.1.1. Creación de la tabla

La tabla es generada automáticamente mediante la instrucción `CREATE TABLE IF NOT EXISTS`. Esto permite crear la estructura durante la primera ejecución y conservarla en las ejecuciones posteriores.

El esquema utilizado es:

```
CREATE TABLE IF NOT EXISTS lecturas_iot (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    fecha_hora TEXT NOT NULL,
    dispositivo TEXT NOT NULL,
    topico TEXT NOT NULL,
    carga_util TEXT NOT NULL
);
```

El archivo `iot_seguro.db` es creado automáticamente por SQLite cuando se establece una conexión mediante `sqlite3.connect()` y el archivo aún no existe.

4.1.2. Inserción de datos

La información recibida se almacena utilizando una consulta SQL parametrizada:

```
INSERT INTO lecturas_iot
(fecha_hora, dispositivo, topico, carga_util)
VALUES (?, ?, ?, ?);
```

Los parámetros utilizados corresponden a:

```
(fecha_hora, dispositivo, topico, payload)
```

El uso de parámetros permite separar la instrucción SQL de los valores recibidos desde la red.

VIGILADA MINEDUCACIÓN

4.1.3. Reglas de integridad de los datos

La tabla implementada establece las siguientes condiciones:

- El campo `id` es único para cada registro.
- El campo `id` se genera automáticamente.
- Los campos `fecha_hora`, `dispositivo`, `topico` y `carga_util` no pueden almacenarse como valores nulos debido a la restricción `NOT NULL`.
- Cada registro corresponde a un mensaje recibido por el sistema.
- La información original recibida se conserva en `carga_util`.

4.2. Componentes de software

El sistema utiliza diferentes librerías de Python para realizar las tareas de comunicación, procesamiento y almacenamiento.

Tabla 2: Librerías utilizadas

Librería	Función
`paho.mqtt.client`	Comunicación con el broker MQTT y recepción de mensajes.
`sqlite3`	Creación, consulta y modificación de la base de datos SQLite.
`datetime`	Obtención y almacenamiento de la fecha y hora del sistema.
`os`	Interacción con funciones y rutas del sistema operativo.
`json`	Procesamiento de información estructurada en formato JSON cuando sea utilizada por los dispositivos.

VIGILADA MINEDUCACIÓN

4.3. Consultas de verificación

Para comprobar el funcionamiento de la base de datos se utilizan diferentes consultas SQL.

4.3.1. Visualización de todos los registros

```
SELECT * FROM lecturas_iot;
```

Esta consulta permite visualizar todos los registros almacenados en la tabla.

4.3.2. Cantidad total de registros

```
SELECT COUNT(*) FROM lecturas_iot;
```

Esta consulta permite determinar la cantidad total de mensajes almacenados.

4.3.3. Consulta de un dispositivo específico

```
SELECT *
FROM lecturas_iot
WHERE dispositivo = 'esp1';
```

Esta consulta permite filtrar la información correspondiente a un dispositivo determinado.

4.4. Exportación y visualización de los datos

Los registros almacenados pueden ser exportados para facilitar su análisis y posterior utilización.

4.4.1. Descarga del archivo de base de datos

La base de datos SQLite se encuentra almacenada en la Raspberry Pi como:

`iot_seguro.db`

El archivo puede descargarse hacia un computador con Windows utilizando el protocolo SCP.

Desde el símbolo del sistema de Windows puede ejecutarse:

```
scp usuario@IP_DE_LA_RASPBERRY:~/iot_seguro.db %USERPROFILE%\
   Desktop
```

Después de introducir las credenciales correspondientes, el archivo será copiado al escritorio del computador.

4.4.2. Exportación a formato CSV

Los datos almacenados también pueden exportarse a formato CSV mediante:

```
sqlite3 -header -csv iot_seguro.db \
"SELECT * FROM lecturas_iot;" > reporte_iot.csv
```

El archivo `reporte_iot.csv` contiene los registros almacenados y puede utilizarse posteriormente para generar gráficas o realizar análisis.

VIGILADA MINEDUCACIÓN

4.4.3. Visualización mediante DB Browser for SQLite

Para facilitar la inspección de la información se puede utilizar **DB Browser for SQLite**.

Esta herramienta permite:

- Visualizar las tablas existentes.
- Consultar los registros almacenados.
- Revisar la estructura de las tablas.
- Ejecutar consultas SQL.
- Filtrar y analizar los registros.
- Exportar información a diferentes formatos.

4.5. Casos de uso

Esta sección presenta situaciones en las que se utiliza la infraestructura desarrollada para recibir y almacenar información procedente de los dispositivos IoT.

4.5.1. Caso de Uso 1: Recepción de datos de un dispositivo ESP32

Un dispositivo ESP32 genera una lectura asociada al proceso de la banda transportadora y publica la información mediante MQTT.

Pasos:

1. El dispositivo ESP32 obtiene o genera la información.
2. Se establece la comunicación con la red Wi-Fi.
3. El dispositivo publica el mensaje en un tópico MQTT.
4. Mosquitto recibe el mensaje.
5. El script Python detecta la llegada del mensaje.
6. Se registra la fecha y hora de recepción.
7. Se identifica el dispositivo y el tópico.
8. La carga útil es almacenada en la tabla `lecturas_iot`.

VIGILADA MINEDUCACIÓN

4.5.2. Caso de Uso 2: Verificación de los datos almacenados

El usuario puede comprobar que los mensajes recibidos desde los dispositivos se almacenan correctamente en la base de datos.

Pasos:

1. Acceder a la Raspberry Pi.
2. Abrir la base de datos `iot_seguro.db`.
3. Consultar la tabla `lecturas_iot`.
4. Ejecutar una consulta SQL.
5. Revisar los registros obtenidos.
6. Exportar los datos a CSV cuando sea necesario.

5. Instalación y configuración

5.1. Requisitos del sistema

Los elementos necesarios para la implementación actual son:

Hardware:

- Raspberry Pi.
- Dispositivos ESP32.
- Sensores y elementos electrónicos asociados al sistema.
- Red inalámbrica Wi-Fi.
- Computador para administración y consulta.

Software:

- Sistema operativo Linux en la Raspberry Pi.
- Python.
- SQLite.
- Broker MQTT Mosquitto.
- Librería `paho-mqtt`.
- DB Browser for SQLite para consulta visual.

VIGILADA MINEDUCACIÓN

5.2. Configuración inicial

La configuración inicial comprende la instalación y puesta en funcionamiento de los servicios necesarios para la recepción y almacenamiento de datos.

El procedimiento general consiste en:

1. Preparar la Raspberry Pi con el sistema operativo Linux.
2. Instalar Python y las dependencias requeridas.
3. Instalar el broker MQTT Mosquitto.
4. Configurar la comunicación MQTT.
5. Crear y ejecutar el script de procesamiento en Python.
6. Crear la base de datos SQLite.
7. Verificar la creación de la tabla `lecturas_iot`.
8. Conectar los dispositivos ESP32 a la red Wi-Fi.
9. Comprobar la recepción de mensajes.
10. Verificar el almacenamiento de los registros.

5.3. Compatibilidad y arquitectura del software

La infraestructura de almacenamiento está basada en herramientas de software que permiten una implementación local y modular.

Python se utiliza como componente de procesamiento y SQLite como sistema de almacenamiento.

Esta arquitectura permite modificar o ampliar posteriormente el sistema para incorporar nuevas variables, dispositivos o mecanismos de consulta.

6. Soporte y mantenimiento

En caso de presentar inconvenientes con el sistema, se recomienda realizar una revisión de los componentes de comunicación, procesamiento y almacenamiento.

La revisión puede comenzar verificando que:

- La Raspberry Pi se encuentre encendida y conectada a la red.
- El broker Mosquitto esté funcionando correctamente.
- El script Python se encuentre ejecutándose.
- Los dispositivos ESP32 estén conectados a la red Wi-Fi.

VIGILADA MINEDUCACIÓN

- Los tópicos MQTT estén siendo utilizados correctamente.
- La base de datos pueda abrirse y recibir nuevos registros.

Para el diagnóstico también pueden utilizarse las consultas SQL de verificación descritas en este documento.

6.1. Actualizaciones

Las futuras actualizaciones del sistema podrán incluir nuevas funciones de procesamiento, incorporación de variables adicionales, nuevos dispositivos IoT y mecanismos de visualización de la información.

También se contempla la integración con la plataforma OT-FORTRESS.

7. Estado actual del sistema

En la etapa actual se encuentran implementados los siguientes elementos:

- Raspberry Pi con sistema operativo Linux.
- Broker Mosquitto MQTT.
- Comunicación mediante MQTT.
- Script de procesamiento desarrollado en Python.
- Base de datos SQLite.
- Archivo `iot_seguro.db`.
- Tabla `lecturas_iot`.
- Registro de fecha y hora de recepción.
- Identificación del dispositivo.
- Registro del tópico MQTT.
- Almacenamiento de la carga útil recibida.
- Consultas de verificación.
- Exportación de información a formato CSV.

VIGILADA MINEDUCACIÓN

7.1. Integración con la plataforma OT-FORTRESS

La plataforma **OT-FORTRESS** se encuentra desarrollada en su versión funcional 1.0.0 como una aplicación de escritorio para entornos Windows.

La solución utiliza Electron, Node.js, Express y SQLite, e incorpora módulos destinados a la supervisión de la arquitectura OT, alertas, red de nodos, dispositivos ESP32, bitácora de eventos y sensores industriales.

En la etapa actual, la base de datos implementada en la Raspberry Pi, `iot_seguro.db`, y la aplicación OT-FORTRESS aún no se encuentran conectadas directamente.

La integración entre ambas soluciones se contempla como una fase posterior del proyecto.

El flujo previsto es:

$$\text{ESP32} \rightarrow \text{MQTT} \rightarrow \text{Raspberry Pi} \rightarrow \text{Python} \rightarrow \text{SQLite} \rightarrow \text{OT-FORTRESS}$$

Una vez realizada esta integración, los datos almacenados podrán ser utilizados por los módulos de supervisión para consulta, visualización, registro de eventos y generación de alertas.

8. Conclusiones

La implementación de la infraestructura de almacenamiento permite establecer un flujo organizado para la recepción y registro de la información generada por los dispositivos IoT asociados al sistema de la banda transportadora.

La utilización de MQTT como protocolo de comunicación, Mosquitto como broker, Python como herramienta de procesamiento y SQLite como sistema de almacenamiento permite centralizar los datos en la Raspberry Pi y mantener un registro estructurado de los mensajes recibidos.

La base de datos `iot_seguro.db` y la tabla `lecturas_iot` proporcionan una estructura para almacenar la fecha y hora de recepción, el dispositivo de origen, el tópico MQTT y la carga útil recibida.

Las consultas SQL y la exportación a formato CSV permiten verificar y analizar la información almacenada, facilitando su utilización en etapas posteriores del proyecto.

Finalmente, la infraestructura desarrollada constituye una base para la integración futura con la plataforma OT-FORTRESS, lo que permitirá ampliar las capacidades del sistema hacia funciones de supervisión, visualización, registro de eventos y monitoreo de los dispositivos y sensores de la banda transportadora.

VIGILADA MINEDUCACIÓN